

Big Data y Machine Learning para Economía Aplicada

Week 08: Redes neuronales: qué son, cuándo usarlas y una aplicación
con imágenes

Eduard F. Martinez Gonzalez, Ph.D.

Departamento de Economía, Universidad Icesi

September 9, 2026

Previously...

- **Los ensambles de árboles** ganan en datos tabulares: encuentran solos las no linealidades y las interacciones. Y la sesión 7 nos dio el kit para **abrir cualquier caja**: importancia por permutación, PDP y ALE, SHAP, con sus tres “no”.

Previously...

- **Los ensambles de árboles** ganan en datos tabulares: encuentran solos las no linealidades y las interacciones. Y la sesión 7 nos dio el kit para **abrir cualquier caja**: importancia por permutación, PDP y ALE, SHAP, con sus tres “no”.
- Dos hilos que hoy convergen: el **gradiente** como motor de aprendizaje (el *boosting* de la sesión 6, la verosimilitud de la sesión 3) y la **regularización** como precio de la flexibilidad (sesión 5).

Previously...

- **Los ensambles de árboles** ganan en datos tabulares: encuentran solos las no linealidades y las interacciones. Y la sesión 7 nos dio el kit para **abrir cualquier caja**: importancia por permutación, PDP y ALE, SHAP, con sus tres “no”.
- Dos hilos que hoy convergen: el **gradiente** como motor de aprendizaje (el *boosting* de la sesión 6, la verosimilitud de la sesión 3) y la **regularización** como precio de la flexibilidad (sesión 5).
- Y una frontera marcada: los árboles pierden cuando la estructura está en la *representación* (imágenes, texto). Ahí no hay columnas que cortar.

Previously...

- **Los ensambles de árboles** ganan en datos tabulares: encuentran solos las no linealidades y las interacciones. Y la sesión 7 nos dio el kit para **abrir cualquier caja**: importancia por permutación, PDP y ALE, SHAP, con sus tres “no”.
- Dos hilos que hoy convergen: el **gradiente** como motor de aprendizaje (el *boosting* de la sesión 6, la verosimilitud de la sesión 3) y la **regularización** como precio de la flexibilidad (sesión 5).
- Y una frontera marcada: los árboles pierden cuando la estructura está en la *representación* (imágenes, texto). Ahí no hay columnas que cortar.

Hoy

Redes neuronales: una composición de regresiones que aprende sus propias variables derivadas; se entrena con gradiente y **retropropagación**; se regulariza con el kit de siempre. Cuándo conviene frente a los árboles, con evidencia y no con fama. Y la aplicación: **pobreza y riqueza desde imágenes satelitales** con una red convolucional preentrenada. Al final, la presentación estudiantil del paper de la sesión 7.

- 1 Qué es una red neuronal
- 2 Cómo se ajusta: gradiente, retropropagación y regularización
- 3 Cuándo usar redes, y cuándo no
- 4 Redes convolucionales: la aplicación con imágenes
- 5 Aplicación guiada: una red contra los árboles, y una CNN que ve mesas
- 6 Interpretación, texto como lectura y cierre

El problema: aprender las variables, no solo los pesos

- Hasta hoy las variables eran **columnas**: las escribimos (sesión 2), las seleccionamos (sesión 5) o el árbol las cortó (sesión 6).

El problema: aprender las variables, no solo los pesos

- Hasta hoy las variables eran **columnas**: las escribimos (sesión 2), las seleccionamos (sesión 5) o el árbol las cortó (sesión 6).
- Una imagen satelital de 224×224 píxeles a color es un vector de 150.528 números sin significado individual; un texto es una secuencia de palabras. No hay columna que cortar ni *spline* que escribir.

El problema: aprender las variables, no solo los pesos

- Hasta hoy las variables eran **columnas**: las escribimos (sesión 2), las seleccionamos (sesión 5) o el árbol las cortó (sesión 6).
- Una imagen satelital de 224×224 píxeles a color es un vector de 150.528 números sin significado individual; un texto es una secuencia de palabras. No hay columna que cortar ni *spline* que escribir.
- El hilo de la sesión 5: **representar es comprimir con sentido**. PCA lo hacía linealmente y sin mirar *y*. Aquí la representación se aprende *para* predecir.

El problema: aprender las variables, no solo los pesos

- Hasta hoy las variables eran **columnas**: las escribimos (sesión 2), las seleccionamos (sesión 5) o el árbol las cortó (sesión 6).
- Una imagen satelital de 224×224 píxeles a color es un vector de 150.528 números sin significado individual; un texto es una secuencia de palabras. No hay columna que cortar ni *spline* que escribir.
- El hilo de la sesión 5: **representar es comprimir con sentido**. PCA lo hacía linealmente y sin mirar y. Aquí la representación se aprende *para* predecir.

Pregunta para la sala

¿Cómo construir una máquina que fabrique sus propias variables derivadas y las use para predecir, todo en un solo ajuste?

El problema: aprender las variables, no solo los pesos

- Hasta hoy las variables eran **columnas**: las escribimos (sesión 2), las seleccionamos (sesión 5) o el árbol las cortó (sesión 6).
- Una imagen satelital de 224×224 píxeles a color es un vector de 150.528 números sin significado individual; un texto es una secuencia de palabras. No hay columna que cortar ni *spline* que escribir.
- El hilo de la sesión 5: **representar es comprimir con sentido**. PCA lo hacía linealmente y sin mirar y. Aquí la representación se aprende *para* predecir.

Pregunta para la sala

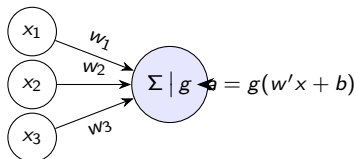
¿Cómo construir una máquina que fabrique sus propias variables derivadas y las use para predecir, todo en un solo ajuste?

Componiendo regresiones: la salida de una capa es la entrada de la siguiente. Cada capa oculta es un conjunto de variables derivadas, aprendidas junto con los pesos que las usan. ISL 10.1 lo dice así: las activaciones son *derived features*.

La neurona: una regresión con actitud

La unidad básica no tiene misterio: una combinación lineal de las entradas seguida de una **función de activación** no lineal g ,

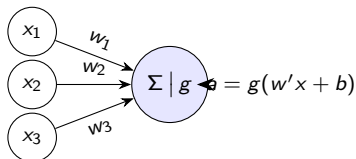
$$a = g(w'x + b).$$



La neurona: una regresión con actitud

La unidad básica no tiene misterio: una combinación lineal de las entradas seguida de una **función de activación** no lineal g ,

$$a = g(w'x + b).$$

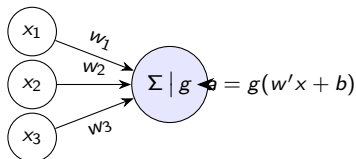


- Con g sigmoide, una neurona **es** la regresión logística de la sesión 3. Nada nuevo todavía.
- La novedad no es la neurona: es **componerlas**.

La neurona: una regresión con actitud

La unidad básica no tiene misterio: una combinación lineal de las entradas seguida de una **función de activación** no lineal g ,

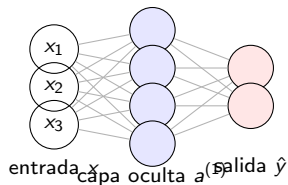
$$a = g(w'x + b).$$



- Con g sigmoide, una neurona **es** la regresión logística de la sesión 3. Nada nuevo todavía.
- La novedad no es la neurona: es **componerlas**.
- Sin la no linealidad no hay negocio: componer funciones lineales da otra función lineal. g es lo que hace que apilar capas agregue capacidad.

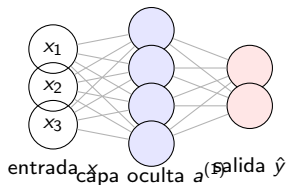
La red *feedforward*: composición de capas

ISL 10.1–10.2



La red *feedforward*: composición de capas

ISL 10.1–10.2

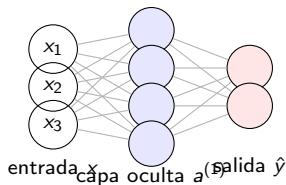


Capa por capa, con $a^{(0)} = x$:

$$a^{(l)} = g(W^{(l)}a^{(l-1)} + b^{(l)}), \quad l = 1, \dots, L-1; \quad \hat{y} = h(W^{(L)}a^{(L-1)} + b^{(L)}).$$

La red *feedforward*: composición de capas

ISL 10.1–10.2

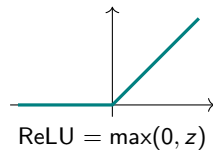
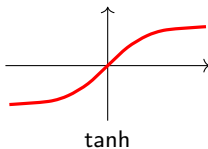
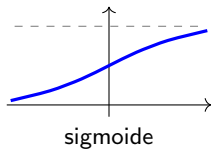


Capa por capa, con $a^{(0)} = x$:

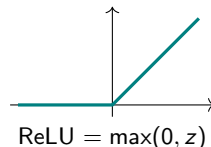
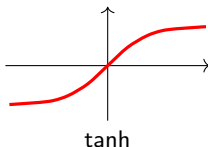
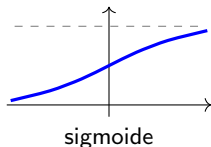
$$a^{(l)} = g(W^{(l)}a^{(l-1)} + b^{(l)}), \quad l = 1, \dots, L-1; \quad \hat{y} = h(W^{(L)}a^{(L-1)} + b^{(L)}).$$

- La red **es** una composición $\hat{y} = f_L \circ \dots \circ f_1(x)$. Profundidad = cuántas funciones componemos; anchura = cuántas unidades por capa.
- **Parámetros**: todos los W, b . Una capa de K unidades sobre p entradas tiene $(p+1)K$ pesos; la red de ISL para dígitos (784 entradas, 256 y 128 ocultas, 10 salidas) tiene 235.146.
- **Salida** h según la tarea: identidad (regresión), sigmoide (binaria), **softmax** (K clases, la multinomial de la sesión 3).

Funciones de activación



Funciones de activación



- **Sigmoide y tanh**: elegantes, pero se **saturan**: en las colas su derivada es casi cero y, con muchas capas, el gradiente se *desvanece* al multiplicarse por casi-ceros.
- **ReLU**: tosca (ni siquiera es diferenciable en 0) pero no se satura en la mitad positiva, es baratísima y entrena mejor: el estándar en capas ocultas.
- **Softmax** en la salida multiclase: convierte K puntajes en probabilidades, $e^{z_k} / \sum_j e^{z_j}$.

Por qué apilar: aproximación universal y profundidad

Teorema de aproximación universal (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera (apéndice A4).

Por qué apilar: aproximación universal y profundidad

Teorema de aproximación universal (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera (apéndice A4).

- Es una garantía de *existencia*: no dice cuántas unidades (pueden ser astronómicas) ni cómo encontrar los pesos.

Por qué apilar: aproximación universal y profundidad

Teorema de aproximación universal (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera (apéndice A4).

- Es una garantía de *existencia*: no dice cuántas unidades (pueden ser astronómicas) ni cómo encontrar los pesos.
- **¿Por qué profundo y no ancho?** La composición **reutiliza**: las capas tempranas aprenden piezas que las tardías combinan. Funciones con estructura composicional se representan con muchos menos parámetros en profundidad que en anchura.

Por qué apilar: aproximación universal y profundidad

Teorema de aproximación universal (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera (apéndice A4).

- Es una garantía de *existencia*: no dice cuántas unidades (pueden ser astronómicas) ni cómo encontrar los pesos.
- **¿Por qué profundo y no ancho?** La composición **reutiliza**: las capas tempranas aprenden piezas que las tardías combinan. Funciones con estructura composicional se representan con muchos menos parámetros en profundidad que en anchura.
- Cada capa oculta es una **representación aprendida** de los datos, de cruda a abstracta. La red no solo predice: fabrica las variables con las que predice.

Por qué apilar: aproximación universal y profundidad

Teorema de aproximación universal (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera (apéndice A4).

- Es una garantía de *existencia*: no dice cuántas unidades (pueden ser astronómicas) ni cómo encontrar los pesos.
- **¿Por qué profundo y no ancho?** La composición **reutiliza**: las capas tempranas aprenden piezas que las tardías combinan. Funciones con estructura composicional se representan con muchos menos parámetros en profundidad que en anchura.
- Cada capa oculta es una **representación aprendida** de los datos, de cruda a abstracta. La red no solo predice: fabrica las variables con las que predice.

El contrato de la flexibilidad

Aproximar todo $\Rightarrow$ sobreajustarlo todo. El teorema garantiza capacidad; la generalización se compra con datos y regularización. La curva U jamás se fue.

- 1 Qué es una red neuronal
- 2 Cómo se ajusta: gradiente, retropropagación y regularización
- 3 Cuándo usar redes, y cuándo no
- 4 Redes convolucionales: la aplicación con imágenes
- 5 Aplicación guiada: una red contra los árboles, y una CNN que ve mesas
- 6 Interpretación, texto como lectura y cierre

La pérdida, y una mala noticia asumida con calma

Nada nuevo en la pérdida:

- Regresión: $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$, el MSE de siempre.
- Clasificación: *cross-entropy*, $-\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$: la máxima verosimilitud de la sesión 3 con signo menos.

La pérdida, y una mala noticia asumida con calma

Nada nuevo en la pérdida:

- Regresión: $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$, el MSE de siempre.
- Clasificación: *cross-entropy*, $-\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$: la máxima verosimilitud de la sesión 3 con signo menos.

La mala noticia: como función de los pesos $\theta = (W, b)$, la pérdida **no es convexa**: valles, mesetas y puntos de silla. Adiós a las garantías del Lasso.

La pérdida, y una mala noticia asumida con calma

Nada nuevo en la pérdida:

- Regresión: $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$, el MSE de siempre.
- Clasificación: *cross-entropy*, $-\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$: la máxima verosimilitud de la sesión 3 con signo menos.

La mala noticia: como función de los pesos $\theta = (W, b)$, la pérdida **no es convexa**: valles, mesetas y puntos de silla. Adiós a las garantías del Lasso. **La sorpresa empírica que sostiene el campo:** el descenso de gradiente *estocástico*, sin garantía de óptimo global, encuentra sistemáticamente soluciones que generalizan bien en redes grandes. Los mínimos “suficientemente buenos” abundan.

La pérdida, y una mala noticia asumida con calma

Nada nuevo en la pérdida:

- Regresión: $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$, el MSE de siempre.
- Clasificación: *cross-entropy*, $-\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$: la máxima verosimilitud de la sesión 3 con signo menos.

La mala noticia: como función de los pesos $\theta = (W, b)$, la pérdida **no es convexa**: valles, mesetas y puntos de silla. Adiós a las garantías del Lasso. **La sorpresa empírica que sostiene el campo:** el descenso de gradiente *estocástico*, sin garantía de óptimo global, encuentra sistemáticamente soluciones que generalizan bien en redes grandes. Los mínimos “suficientemente buenos” abundan.

Actitud del curso

Óptimo local + buen protocolo (inicializaciones, monitoreo con validación, regularización) = herramienta utilizable. La perfección teórica no es requisito para la utilidad práctica; la prueba honesta, sí.

Descenso de gradiente estocástico (SGD)

La regla de siempre, ahora sobre miles o millones de parámetros:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

Descenso de gradiente estocástico (SGD)

La regla de siempre, ahora sobre miles o millones de parámetros:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

- **Mini-lotes:** el gradiente se calcula con 32 a 256 observaciones por paso, no con todas: ruidoso pero barato, y el ruido ayuda a escapar de valles malos. Una **época** es una pasada por los datos.
- **η , la tasa de aprendizaje:** la perilla delicada. Grande diverge; chica no llega. **Adam** (Kingma & Ba, 2015) adapta el paso por parámetro y es el *default* moderno (apéndice A2).
- Se entrena por épocas **monitoreando la validación:** la curva que sigue decide cuándo parar.

Descenso de gradiente estocástico (SGD)

La regla de siempre, ahora sobre miles o millones de parámetros:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

- **Mini-lotes:** el gradiente se calcula con 32 a 256 observaciones por paso, no con todas: ruidoso pero barato, y el ruido ayuda a escapar de valles malos. Una **época** es una pasada por los datos.
- **η , la tasa de aprendizaje:** la perilla delicada. Grande diverge; chica no llega. **Adam** (Kingma & Ba, 2015) adapta el paso por parámetro y es el *default* moderno (apéndice A2).
- Se entrena por épocas **monitoreando la validación:** la curva que sigue decide cuándo parar.

Déjà vu del *boosting*

En la sesión 6 hicimos descenso de gradiente en el espacio de *funciones*: un árbol por paso. Aquí es en el espacio de *parámetros*: un mini-lote por paso. Mismo motor, distinto chasis.

Retropropagación: la regla de la cadena, administrada

ISL 10.7.1; la derivación completa en el apéndice A1

El problema computacional: $\nabla_{\theta} L$ tiene miles de componentes. ¿Cómo calcularlos sin morir?

Retropropagación: la regla de la cadena, administrada

ISL 10.7.1; la derivación completa en el apéndice A1

El problema computacional: $\nabla_{\theta} L$ tiene miles de componentes. ¿Cómo calcularlos sin morir? **La respuesta:** la red es una composición, así que la derivada de la pérdida respecto a *cualquier* peso es un producto de derivadas locales a lo largo del camino hasta la salida. Esos productos **comparten factores**: computando de la salida hacia atrás, cada factor se calcula una sola vez y se reutiliza.

Retropropagación: la regla de la cadena, administrada

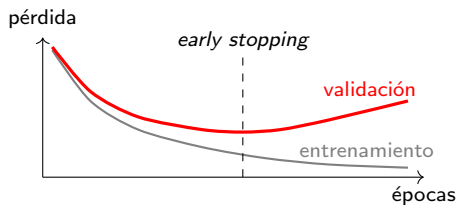
ISL 10.7.1; la derivación completa en el apéndice A1

El problema computacional: $\nabla_{\theta} L$ tiene miles de componentes. ¿Cómo calcularlos sin morir? **La respuesta:** la red es una composición, así que la derivada de la pérdida respecto a *cualquier* peso es un producto de derivadas locales a lo largo del camino hasta la salida. Esos productos **comparten factores**: computando de la salida hacia atrás, cada factor se calcula una sola vez y se reutiliza.

- Resultado: el gradiente completo cuesta lo mismo que **una pasada hacia adelante más una hacia atrás**, no una pasada por parámetro. Esa economía es lo que hace entrenables las redes grandes.
- El objeto que viaja hacia atrás es el “error” de cada capa, $\delta^{(l)} = \partial L / \partial z^{(l)}$; en la salida, con pérdida cuadrática, $\delta^{(L)} = \hat{y} - y$: **el residuo, otra vez**. El patrón: δ de una capa = (δ de la siguiente $\times$ pesos) $\odot$ derivada de la activación.
- En la práctica nadie deriva a mano: **diferenciación automática** (Keras, PyTorch). Ustedes lo harán una vez con números: es material del Problem Set 2.

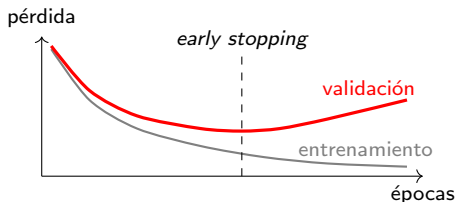
Regularizar redes: el kit de siempre, con otros nombres

ISL 10.7.2–10.7.3



Regularizar redes: el kit de siempre, con otros nombres

ISL 10.7.2–10.7.3



- **Weight decay** (L_2): penalizar $\sum \|W\|^2$: Ridge (sesión 5) con otro nombre.
- **Early stopping**: parar cuando la pérdida de validación deja de bajar: elegir el mínimo de la curva U en vivo, con las épocas como eje de complejidad. Es la perilla que *siempre* se usa.
- **Dropout** (Srivastava et al., 2014): en cada paso, apagar al azar una fracción de unidades; la red no puede depender de ninguna en particular. En el fondo, un ensamble implícito: el espíritu del bosque.
- **Aumentar datos**: en imágenes, rotar, recortar, reflejar. El regularizador más honesto: más variación invariante.

Los hiperparámetros de una red, en una tabla

Perilla	Qué controla	Rango típico	Se elige por
capas y unidades	capacidad	1–3 capas; 16–256 unid.	validación
activación	no linealidad	ReLU	<i>default</i>
tasa de aprendizaje η	tamaño del paso	10^{-4} a 10^{-2}	validación
tamaño del mini-lote	ruido del gradiente	32 a 256	fijo
épocas	cuándo parar	hasta 500	<i>early stopping</i>
<i>dropout</i>	ensamble implícito	0,1 a 0,5	validación
<i>weight decay</i>	Ridge sobre los pesos	10^{-5} a 10^{-3}	validación
semilla	inicialización, orden de lotes	varias	se <i>promedia</i>

Los hiperparámetros de una red, en una tabla

Perilla	Qué controla	Rango típico	Se elige por
capas y unidades	capacidad	1–3 capas; 16–256 unid.	validación
activación	no linealidad	ReLU	<i>default</i>
tasa de aprendizaje η	tamaño del paso	10^{-4} a 10^{-2}	validación
tamaño del mini-lote	ruido del gradiente	32 a 256	fijo
épocas	cuándo parar	hasta 500	<i>early stopping</i>
<i>dropout</i>	ensamble implícito	0,1 a 0,5	validación
<i>weight decay</i>	Ridge sobre los pesos	10^{-5} a 10^{-3}	validación
semilla	inicialización, orden de lotes	varias	se <i>promedia</i>

- **Las redes sí necesitan escala:** normalizar los predictores dentro del *pipeline* (regla de oro #3), a diferencia de los árboles.
- **La semilla es un hiperparámetro que no se elige:** el mismo modelo con otra inicialización da otro resultado. Se reporta la media y la dispersión sobre varias semillas, o se promedian las redes.
- La búsqueda es cara: con CV de 5 pliegues cada configuración son cinco entrenamientos. Búsqueda aleatoria (sesión 4) y pocas perillas a la vez.

- 1 Qué es una red neuronal
- 2 Cómo se ajusta: gradiente, retropropagación y regularización
- 3 Cuándo usar redes, y cuándo no
- 4 Redes convolucionales: la aplicación con imágenes
- 5 Aplicación guiada: una red contra los árboles, y una CNN que ve mesas
- 6 Interpretación, texto como lectura y cierre

Datos tabulares: el desafío de los árboles

ISL 10.6: cuando la ganancia es pequeña, gana el modelo simple

- **ISL 10.6:** en los salarios de *Hitters*, la red y el Lasso quedan casi empatados en la prueba. La navaja de Occam: si la red no gana claramente, se usa el modelo que se entiende.

Datos tabulares: el desafío de los árboles

ISL 10.6: cuando la ganancia es pequeña, gana el modelo simple

- **ISL 10.6:** en los salarios de *Hitters*, la red y el Lasso quedan casi empatados en la prueba. La navaja de Occam: si la red no gana claramente, se usa el modelo que se entiende.
- **Grinsztajn, Oyallon & Varoquaux (2022):** en 45 conjuntos tabulares de tamaño medio, los ensambles de árboles superan a las redes, incluso afinadas con presupuesto grande. Las redes prefieren funciones suaves, sufren con variables irrelevantes y dependen de la escala.

Datos tabulares: el desafío de los árboles

ISL 10.6: cuando la ganancia es pequeña, gana el modelo simple

- **ISL 10.6:** en los salarios de *Hitters*, la red y el Lasso quedan casi empatados en la prueba. La navaja de Occam: si la red no gana claramente, se usa el modelo que se entiende.
- **Grinsztajn, Oyallon & Varoquaux (2022):** en 45 conjuntos tabulares de tamaño medio, los ensambles de árboles superan a las redes, incluso afinadas con presupuesto grande. Las redes prefieren funciones suaves, sufren con variables irrelevantes y dependen de la escala.
- **En economía:** Medeiros et al. (2021), el bosque gana en inflación; Gelvez et al. (2026), el bosque; Gu, Kelly & Xiu (2020), redes y árboles ganan con millones de observaciones acción-mes, y la ganancia viene de las interacciones.

Datos tabulares: el desafío de los árboles

ISL 10.6: cuando la ganancia es pequeña, gana el modelo simple

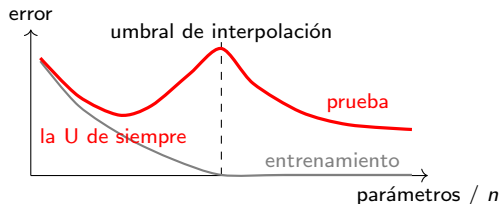
- **ISL 10.6:** en los salarios de *Hitters*, la red y el Lasso quedan casi empatados en la prueba. La navaja de Occam: si la red no gana claramente, se usa el modelo que se entiende.
- **Grinsztajn, Oyallon & Varoquaux (2022):** en 45 conjuntos tabulares de tamaño medio, los ensambles de árboles superan a las redes, incluso afinadas con presupuesto grande. Las redes prefieren funciones suaves, sufren con variables irrelevantes y dependen de la escala.
- **En economía:** Medeiros et al. (2021), el bosque gana en inflación; Gelvez et al. (2026), el bosque; Gu, Kelly & Xiu (2020), redes y árboles ganan con millones de observaciones acción-mes, y la ganancia viene de las interacciones.

La regla práctica

Con n de miles y columnas con sentido, XGBoost afinado es el techo razonable y la red rara vez lo supera. La red paga con **estructura** (imágenes, texto), con **escala**, o para **transferir** una representación ya aprendida.

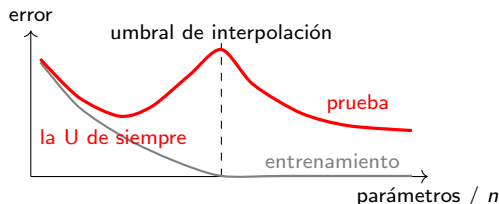
Interpolación y doble descenso, en una diapositiva

ISL 10.8: la curva U tiene un segundo acto



Interpolación y doble descenso, en una diapositiva

ISL 10.8: la curva U tiene un segundo acto



- A la izquierda del umbral, la curva U de la sesión 1. En el umbral, la red **interpola**: error de entrenamiento cero, error de prueba máximo. Más allá, con muchísimos más parámetros que datos, el error de prueba **vuelve a bajar**: entre todas las soluciones que interpolan, SGD tiende a elegir la “más suave”.
- **No es una licencia**: la regularización explícita sigue mejorando, y a las escalas de este curso (miles de mesas, decenas de columnas) trabajamos a la izquierda del umbral. Es la explicación de por qué las redes gigantes no colapsan, no una receta para sus proyectos.

- 1 Qué es una red neuronal
- 2 Cómo se ajusta: gradiente, retropropagación y regularización
- 3 Cuándo usar redes, y cuándo no
- 4 Redes convolucionales: la aplicación con imágenes**
- 5 Aplicación guiada: una red contra los árboles, y una CNN que ve mesas
- 6 Interpretación, texto como lectura y cierre

Por qué una red densa no puede ver

Una imagen modesta de 200×200 píxeles a color son 120.000 entradas.

Por qué una red densa no puede ver

Una imagen modesta de 200×200 píxeles a color son 120.000 entradas.

- Una primera capa densa de 1.000 unidades tendría **120 millones** de pesos en una sola capa: sobreajuste garantizado y cómputo absurdo.

Por qué una red densa no puede ver

Una imagen modesta de 200×200 píxeles a color son 120.000 entradas.

- Una primera capa densa de 1.000 unidades tendría **120 millones** de pesos en una sola capa: sobreajuste garantizado y cómputo absurdo.
- Peor: la red densa **ignora la estructura espacial**. Trata el píxel $(1, 1)$ y el $(200, 200)$ como si su relación fuera la de dos vecinos; si el techo se corre diez píxeles, para la red es una entrada completamente nueva.

Por qué una red densa no puede ver

Una imagen modesta de 200×200 píxeles a color son 120.000 entradas.

- Una primera capa densa de 1.000 unidades tendría **120 millones** de pesos en una sola capa: sobreajuste garantizado y cómputo absurdo.
- Peor: la red densa **ignora la estructura espacial**. Trata el píxel (1,1) y el (200,200) como si su relación fuera la de dos vecinos; si el techo se corre diez píxeles, para la red es una entrada completamente nueva.

Las dos ideas que arreglan todo (LeCun et al., 1998; ISL 10.3):

- 1 **Localidad**: los patrones visuales son locales; un borde vive en una ventanita.
- 2 **Invarianza a traslación**: un borde es un borde esté donde esté, así que se usan **los mismos pesos** en toda la imagen.

Por qué una red densa no puede ver

Una imagen modesta de 200×200 píxeles a color son 120.000 entradas.

- Una primera capa densa de 1.000 unidades tendría **120 millones** de pesos en una sola capa: sobreajuste garantizado y cómputo absurdo.
- Peor: la red densa **ignora la estructura espacial**. Trata el píxel (1, 1) y el (200, 200) como si su relación fuera la de dos vecinos; si el techo se corre diez píxeles, para la red es una entrada completamente nueva.

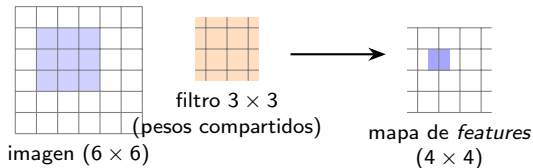
Las dos ideas que arreglan todo (LeCun et al., 1998; ISL 10.3):

- 1 **Localidad**: los patrones visuales son locales; un borde vive en una ventanita.
- 2 **Invarianza a traslación**: un borde es un borde esté donde esté, así que se usan **los mismos pesos** en toda la imagen.

Juntas se llaman **convolución**, y convierten los 120 millones de pesos en unas decenas por filtro.

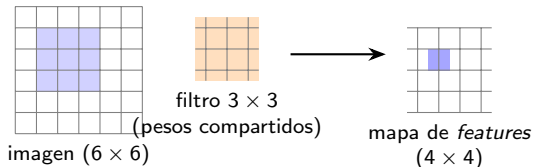
La convolución: un filtro que recorre la imagen

ISL 10.3.1–10.3.2



La convolución: un filtro que recorre la imagen

ISL 10.3.1–10.3.2



- El filtro, una matriz pequeña de pesos, se **desliza** por la imagen; en cada posición, el producto punto ventana–filtro da un número del mapa de salida.
- Cada filtro aprende a **detectar un patrón** (un borde vertical, una esquina, una textura); su mapa dice *dónde* apareció. Una capa convolucional son varios filtros en paralelo (apéndice A3).
- **Pooling**: quedarse con el máximo de cada bloque 2 × 2; reduce la resolución, gana robustez a desplazamientos pequeños y abarata lo que sigue.

La arquitectura CNN y el *transfer learning*

ISL 10.3.3–10.3.5

El patrón: [convolución $\rightarrow$ ReLU $\rightarrow$ *pooling*] varias veces $\rightarrow$ capas densas $\rightarrow$ salida. Las capas tempranas aprenden **bordes**; las medias, **texturas y partes** (techos, vías, cultivos); las tardías, **objetos y escenas**. La composición de la Parte 1, hecha visible.

La arquitectura CNN y el *transfer learning*

ISL 10.3.3–10.3.5

El patrón: [convolución → ReLU → *pooling*] varias veces → capas densas → salida. Las capas tempranas aprenden **bordes**; las medias, **texturas y partes** (techos, vías, cultivos); las tardías, **objetos y escenas**. La composición de la Parte 1, hecha visible. **El momento histórico:** AlexNet gana ImageNet (2012) por paliza: profundidad, ReLU, *dropout* y GPU. Desde entonces la visión por computadora es aprendizaje profundo.

La arquitectura CNN y el *transfer learning*

ISL 10.3.3–10.3.5

El patrón: [convolución → ReLU → *pooling*] varias veces → capas densas → salida. Las capas tempranas aprenden **bordes**; las medias, **texturas y partes** (techos, vías, cultivos); las tardías, **objetos y escenas**. La composición de la Parte 1, hecha visible. **El momento histórico:** AlexNet gana ImageNet (2012) por paliza: profundidad, ReLU, *dropout* y GPU. Desde entonces la visión por computadora es aprendizaje profundo.

Transfer learning: el truco que nos habilita

Una CNN entrenada con millones de imágenes genéricas ya sabe ver bordes, texturas y partes; eso es **universal**. Receta: tomar la red preentrenada (ResNet50 en ISL 10.3.5), **congelar** sus capas y usar su penúltima capa como 2.048 *variables derivadas*; encima, un Ridge o un bosque con *nuestros* datos (miles, no millones). Reentrenar las últimas capas solo si sobran datos. Así se hace visión con presupuesto de economista.

La arquitectura CNN y el *transfer learning*

ISL 10.3.3–10.3.5

El patrón: [convolución $\rightarrow$ ReLU $\rightarrow$ *pooling*] varias veces $\rightarrow$ capas densas $\rightarrow$ salida. Las capas tempranas aprenden **bordes**; las medias, **texturas y partes** (techos, vías, cultivos); las tardías, **objetos y escenas**. La composición de la Parte 1, hecha visible. **El momento histórico:** AlexNet gana ImageNet (2012) por paliza: profundidad, ReLU, *dropout* y GPU. Desde entonces la visión por computadora es aprendizaje profundo.

Transfer learning: el truco que nos habilita

Una CNN entrenada con millones de imágenes genéricas ya sabe ver bordes, texturas y partes; eso es **universal**. Receta: tomar la red preentrenada (ResNet50 en ISL 10.3.5), **congelar** sus capas y usar su penúltima capa como 2.048 *variables derivadas*; encima, un Ridge o un bosque con *nuestros* datos (miles, no millones). Reentrenar las últimas capas solo si sobran datos. Así se hace visión con presupuesto de economista.

- El *pipeline* de la sesión 4 no cambia: la red congelada es un paso de preprocesamiento; el Ridge se afina con CV; la prueba se usa una vez.

Jean et al. (2016, *Science*): pobreza vista desde el cielo

El problema: en el mundo en desarrollo las encuestas de hogares son escasas, caras y poco frecuentes, justo donde más se necesita medir la pobreza.

Jean et al. (2016, *Science*): pobreza vista desde el cielo

El problema: en el mundo en desarrollo las encuestas de hogares son escasas, caras y poco frecuentes, justo donde más se necesita medir la pobreza. **La solución** (*transfer learning* doblemente inteligente):

- 1 Partir de una CNN preentrenada en imágenes genéricas (ImageNet).
- 2 Reentrenarla para predecir **luces nocturnas** desde imágenes **diurnas**: un proxy conocido de actividad económica con datos de sobra, sin una sola encuesta.
- 3 En ese proceso la red aprende sola a reconocer techos, vías, cultivos, urbanización.
- 4 Usar esas *features* en un Ridge para predecir el **consumo y la riqueza** de las encuestas disponibles, con CV.

Jean et al. (2016, *Science*): pobreza vista desde el cielo

El problema: en el mundo en desarrollo las encuestas de hogares son escasas, caras y poco frecuentes, justo donde más se necesita medir la pobreza. **La solución** (*transfer learning* doblemente inteligente):

- 1 Partir de una CNN preentrenada en imágenes genéricas (ImageNet).
- 2 Reentrenarla para predecir **luces nocturnas** desde imágenes **diurnas**: un proxy conocido de actividad económica con datos de sobra, sin una sola encuesta.
- 3 En ese proceso la red aprende sola a reconocer techos, vías, cultivos, urbanización.
- 4 Usar esas *features* en un Ridge para predecir el **consumo y la riqueza** de las encuestas disponibles, con CV.

Resultado: en cinco países africanos explica del **37 % al 55 %** de la variación del consumo local y hasta el **75 %** de la riqueza en activos; supera a las luces nocturnas solas.

Jean et al. (2016, *Science*): pobreza vista desde el cielo

El problema: en el mundo en desarrollo las encuestas de hogares son escasas, caras y poco frecuentes, justo donde más se necesita medir la pobreza. **La solución** (*transfer learning* doblemente inteligente):

- 1 Partir de una CNN preentrenada en imágenes genéricas (ImageNet).
- 2 Reentrenarla para predecir **luces nocturnas** desde imágenes **diurnas**: un proxy conocido de actividad económica con datos de sobra, sin una sola encuesta.
- 3 En ese proceso la red aprende sola a reconocer techos, vías, cultivos, urbanización.
- 4 Usar esas *features* en un Ridge para predecir el **consumo y la riqueza** de las encuestas disponibles, con CV.

Resultado: en cinco países africanos explica del **37 % al 55 %** de la variación del consumo local y hasta el **75 %** de la riqueza en activos; supera a las luces nocturnas solas.

Por qué es la aplicación perfecta del curso

Transfer learning + *pipeline* con CV y prueba (sesión 4) + un proxy validado contra el terreno (sesión 7), para un problema de medición real.

El ecosistema: satélites, luces y política pública

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012): crecimiento donde las cuentas nacionales son débiles. En Gelvez et al. (2026) la luminosidad ya es una variable del bloque de infraestructura.

El ecosistema: satélites, luces y política pública

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012): crecimiento donde las cuentas nacionales son débiles. En Gelvez et al. (2026) la luminosidad ya es una variable del bloque de infraestructura.
- **Riqueza a escala continental** (Yeh et al., 2020): Jean et al. extendido a toda África subsahariana, validado contra encuestas y en el tiempo.

El ecosistema: satélites, luces y política pública

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012): crecimiento donde las cuentas nacionales son débiles. En Gelvez et al. (2026) la luminosidad ya es una variable del bloque de infraestructura.
- **Riqueza a escala continental** (Yeh et al., 2020): Jean et al. extendido a toda África subsahariana, validado contra encuestas y en el tiempo.
- **Vivienda y ciudades:** Glaeser, Kincaid & Naik (2018) predicen precios en Boston desde fotos de fachada: las imágenes añaden poco frente a la ubicación, pero superan al tasador presencial. Catastros, expansión informal, deforestación: la misma caja.

El ecosistema: satélites, luces y política pública

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012): crecimiento donde las cuentas nacionales son débiles. En Gelvez et al. (2026) la luminosidad ya es una variable del bloque de infraestructura.
- **Riqueza a escala continental** (Yeh et al., 2020): Jean et al. extendido a toda África subsahariana, validado contra encuestas y en el tiempo.
- **Vivienda y ciudades:** Glaeser, Kincaid & Naik (2018) predicen precios en Boston desde fotos de fachada: las imágenes añaden poco frente a la ubicación, pero superan al tasador presencial. Catastros, expansión informal, deforestación: la misma caja.
- **Focalización:** mapas de pobreza de alta resolución donde las encuestas no llegan (Blumenstock et al., 2015, con telefonía).

El ecosistema: satélites, luces y política pública

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012): crecimiento donde las cuentas nacionales son débiles. En Gelvez et al. (2026) la luminosidad ya es una variable del bloque de infraestructura.
- **Riqueza a escala continental** (Yeh et al., 2020): Jean et al. extendido a toda África subsahariana, validado contra encuestas y en el tiempo.
- **Vivienda y ciudades:** Glaeser, Kincaid & Naik (2018) predicen precios en Boston desde fotos de fachada: las imágenes añaden poco frente a la ubicación, pero superan al tasador presencial. Catastros, expansión informal, deforestación: la misma caja.
- **Focalización:** mapas de pobreza de alta resolución donde las encuestas no llegan (Blumenstock et al., 2015, con telefonía).

La advertencia de medición

Lo predicho es un **proxy**: hereda los sesgos de su entrenamiento (¿qué pobreza *no* se ve desde arriba?) y se degrada fuera del dominio. Validar contra el terreno, con error por subgrupos, no es opcional.

- 1 Qué es una red neuronal
- 2 Cómo se ajusta: gradiente, retropropagación y regularización
- 3 Cuándo usar redes, y cuándo no
- 4 Redes convolucionales: la aplicación con imágenes
- 5 Aplicación guiada: una red contra los árboles, y una CNN que ve mesas
- 6 Interpretación, texto como lectura y cierre

Datos y preguntas

Parte A, tabular: la participación del ganador en 2022, primera vuelta, con *la misma partición y la misma prueba* de la sesión 6. Una red densa (una y tres capas, con *dropout*, *early stopping* y varias semillas) contra OLS, el bosque y XGBoost. Métricas: RMSE, MAE, R^2 con intervalos.

Datos y preguntas

Parte A, tabular: la participación del ganador en 2022, primera vuelta, con *la misma partición y la misma prueba* de la sesión 6. Una red densa (una y tres capas, con *dropout*, *early stopping* y varias semillas) contra OLS, el bosque y XGBoost. Métricas: RMSE, MAE, R^2 con intervalos. **Parte B, imágenes:** una imagen satelital diurna por mesa (el entorno censal de la sesión 3). Una ResNet50 congelada convierte cada imagen en 2.048 variables; encima, un Ridge (sesión 5) para predecir la **luminosidad nocturna** y la **proporción de estrato bajo** de la mesa. Es Jean et al. en pequeño.

Datos y preguntas

Parte A, tabular: la participación del ganador en 2022, primera vuelta, con *la misma partición y la misma prueba* de la sesión 6. Una red densa (una y tres capas, con *dropout*, *early stopping* y varias semillas) contra OLS, el bosque y XGBoost. Métricas: RMSE, MAE, R^2 con intervalos. **Parte B, imágenes:** una imagen satelital diurna por mesa (el entorno censal de la sesión 3). Una ResNet50 congelada convierte cada imagen en 2.048 variables; encima, un Ridge (sesión 5) para predecir la **luminosidad nocturna** y la **proporción de estrato bajo** de la mesa. Es Jean et al. en pequeño. **Protocolo:** normalización dentro del *pipeline*; validación por **bloques de municipio** (sesión 4) porque las imágenes vecinas se parecen; *baselines*: la media, y para la luminosidad, un OLS con las variables censales. La prueba, una vez.

Datos y preguntas

Parte A, tabular: la participación del ganador en 2022, primera vuelta, con *la misma partición y la misma prueba* de la sesión 6. Una red densa (una y tres capas, con *dropout*, *early stopping* y varias semillas) contra OLS, el bosque y XGBoost. Métricas: RMSE, MAE, R^2 con intervalos. **Parte B, imágenes:** una imagen satelital diurna por mesa (el entorno censal de la sesión 3). Una ResNet50 congelada convierte cada imagen en 2.048 variables; encima, un Ridge (sesión 5) para predecir la **luminosidad nocturna** y la **proporción de estrato bajo** de la mesa. Es Jean et al. en pequeño. **Protocolo:** normalización dentro del *pipeline*; validación por **bloques de municipio** (sesión 4) porque las imágenes vecinas se parecen; *baselines*: la media, y para la luminosidad, un OLS con las variables censales. La prueba, una vez.

Las preguntas de hoy

¿Le gana una red a XGBoost en una tabla de decenas de columnas? ¿Cuánta información sobre el territorio hay en una imagen, y cuánta de esa ya estaba en el censo?

La tabla: la red contra los árboles, una misma prueba

Participación del ganador, 2022, primera vuelta: mesas de prueba

Modelo	RMSE (pp)	MAE (pp)	R^2 de prueba
OLS, todas las características	en clase	en clase	0,578
<i>Random forest</i> afinado	14,69	10,90	0,662
XGBoost afinado	en clase	en clase	en clase
Red densa, 1 capa (64), 5 semillas	en clase	en clase	en clase
Red densa, 3 capas, <i>dropout</i> 0,2	en clase	en clase	en clase

Cifras publicadas: Gelvez et al. (2026), Tablas 1 y 6. Las celdas en gris se calculan con el código de la sesión; para las redes se reporta la media sobre semillas y su desviación.

La tabla: la red contra los árboles, una misma prueba

Participación del ganador, 2022, primera vuelta: mesas de prueba

Modelo	RMSE (pp)	MAE (pp)	R^2 de prueba
OLS, todas las características	en clase	en clase	0,578
<i>Random forest</i> afinado	14,69	10,90	0,662
XGBoost afinado	en clase	en clase	en clase
Red densa, 1 capa (64), 5 semillas	en clase	en clase	en clase
Red densa, 3 capas, <i>dropout</i> 0,2	en clase	en clase	en clase

Cifras publicadas: Gelvez et al. (2026), Tablas 1 y 6. Las celdas en gris se calculan con el código de la sesión; para las redes se reporta la media sobre semillas y su desviación.

- **Qué esperar:** la red bien afinada debería acercarse al bosque, no superarlo con claridad; si queda por debajo de OLS, casi siempre es la escala, la tasa de aprendizaje o parar tarde.

La tabla: la red contra los árboles, una misma prueba

Participación del ganador, 2022, primera vuelta: mesas de prueba

Modelo	RMSE (pp)	MAE (pp)	R^2 de prueba
OLS, todas las características	en clase	en clase	0,578
<i>Random forest</i> afinado	14,69	10,90	0,662
XGBoost afinado	en clase	en clase	en clase
Red densa, 1 capa (64), 5 semillas	en clase	en clase	en clase
Red densa, 3 capas, <i>dropout</i> 0,2	en clase	en clase	en clase

Cifras publicadas: Gelvez et al. (2026), Tablas 1 y 6. Las celdas en gris se calculan con el código de la sesión; para las redes se reporta la media sobre semillas y su desviación.

- **Qué esperar:** la red bien afinada debería acercarse al bosque, no superarlo con claridad; si queda por debajo de OLS, casi siempre es la escala, la tasa de aprendizaje o parar tarde.
- **Qué reportar:** la dispersión entre semillas junto al intervalo *bootstrap*: si son del mismo tamaño, la “ganancia” de una arquitectura sobre otra es ruido.

La tabla: la red contra los árboles, una misma prueba

Participación del ganador, 2022, primera vuelta: mesas de prueba

Modelo	RMSE (pp)	MAE (pp)	R^2 de prueba
OLS, todas las características	en clase	en clase	0,578
<i>Random forest</i> afinado	14,69	10,90	0,662
XGBoost afinado	en clase	en clase	en clase
Red densa, 1 capa (64), 5 semillas	en clase	en clase	en clase
Red densa, 3 capas, <i>dropout</i> 0,2	en clase	en clase	en clase

Cifras publicadas: Gelvez et al. (2026), Tablas 1 y 6. Las celdas en gris se calculan con el código de la sesión; para las redes se reporta la media sobre semillas y su desviación.

- **Qué esperar:** la red bien afinada debería acercarse al bosque, no superarlo con claridad; si queda por debajo de OLS, casi siempre es la escala, la tasa de aprendizaje o parar tarde.
- **Qué reportar:** la dispersión entre semillas junto al intervalo *bootstrap*: si son del mismo tamaño, la “ganancia” de una arquitectura sobre otra es ruido.

Parte B (imágenes → luminosidad): R^2 de prueba de la CNN congelada + Ridge, en clase; *baseline* censal, en clase. Referencia: Jean et al. (2016) explican 37 a 55 % del consumo.

Tres figuras

- 1 **Curvas de entrenamiento y validación por época**, una por semilla. Qué buscar: dónde se separa la validación del entrenamiento (ahí para el *early stopping*) y cuánto difieren las semillas. Es la figura de la regularización, con datos.

Tres figuras

- 1 **Curvas de entrenamiento y validación por época**, una por semilla. Qué buscar: dónde se separa la validación del entrenamiento (ahí para el *early stopping*) y cuánto difieren las semillas. Es la figura de la regularización, con datos.
- 2 **Predicho vs. observado** en la prueba: red y bosque lado a lado. Qué buscar: la misma compresión hacia la media de la sesión 6; si la red comprime menos en las colas, es porque extrapola de forma más suave.

Tres figuras

- 1 **Curvas de entrenamiento y validación por época**, una por semilla. Qué buscar: dónde se separa la validación del entrenamiento (ahí para el *early stopping*) y cuánto difieren las semillas. Es la figura de la regularización, con datos.
- 2 **Predicho vs. observado** en la prueba: red y bosque lado a lado. Qué buscar: la misma compresión hacia la media de la sesión 6; si la red comprime menos en las colas, es porque extrapola de forma más suave.
- 3 **Lo que la CNN ve**: las diez imágenes con mayor y menor luminosidad predicha, y para una de ellas el mapa de saliencia (qué píxeles mueven la predicción). Qué buscar: techos, vías y densidad en las altas; vegetación y dispersión en las bajas. Y la advertencia de la sesión 7: la saliencia explica el modelo, no el territorio.

Tres figuras

- 1 **Curvas de entrenamiento y validación por época**, una por semilla. Qué buscar: dónde se separa la validación del entrenamiento (ahí para el *early stopping*) y cuánto difieren las semillas. Es la figura de la regularización, con datos.
- 2 **Predicho vs. observado** en la prueba: red y bosque lado a lado. Qué buscar: la misma compresión hacia la media de la sesión 6; si la red comprime menos en las colas, es porque extrapola de forma más suave.
- 3 **Lo que la CNN ve**: las diez imágenes con mayor y menor luminosidad predicha, y para una de ellas el mapa de saliencia (qué píxeles mueven la predicción). Qué buscar: techos, vías y densidad en las altas; vegetación y dispersión en las bajas. Y la advertencia de la sesión 7: la saliencia explica el modelo, no el territorio.

Y una cuarta, barata

El error de la Parte B por municipio, en un mapa: si los fallos se agrupan, la validación aleatoria habría mentido (sesión 4).

Cómo se ve en R (I): una red densa con keras

```
library(keras); library(tidymodels); set.seed(2026)
rec <- recipe(share ~ ., data = train) |>
  step_impute_mean(all_numeric_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors()) |> prep() # escala
x_tr <- bake(rec, train, all_predictors(), composition = "matrix")
x_te <- bake(rec, test, all_predictors(), composition = "matrix")
red <- keras_model_sequential() |>
  layer_dense(64, "relu", input_shape = ncol(x_tr)) |>
  layer_dropout(0.2) |> layer_dense(32, "relu") |>
  layer_dense(1) # salida lineal
red |> compile(loss = "mse", optimizer = optimizer_adam(1e-3))
h <- red |> fit(x_tr, train$share, epochs = 200, batch_size = 128,
  validation_split = 0.2, # parar con validación
  callbacks = callback_early_stopping(patience = 15,
    restore_best_weights = TRUE))
rmse_vec(test$share, predict(red, x_te)[, 1]) # prueba, 1 vez
```

- La receta normaliza *dentro* del entrenamiento; `validation_split` aparta el 20 % para el *early stopping*; `plot(h)` dibuja las curvas. Repetir con varias semillas y promediar.

Cómo se ve en R (II): la CNN preentrenada como preprocesamiento

```
base <- application_resnet50(weights = "imagenet",
  include_top = FALSE, pooling = "avg")      # congelada
tiles <- array(0, c(n, 224, 224, 3))        # una imagen por mesa
for (i in seq_len(n)) tiles[i, , , ] <- image_to_array(
  image_load(rutas[i], target_size = c(224, 224)))
feat <- predict(base, imagenet_preprocess_input(tiles)) # n x 2048
datos <- bind_cols(mesas_img, as_tibble(feat))
rec_i <- recipe(luminosidad ~ ., data = datos) |>
  step_normalize(all_numeric_predictors())
ridge <- linear_reg(penalty = tune(), mixture = 0) |>
  set_engine("glmnet")                       # Ridge sobre 2.048 columnas
folds <- group_vfold_cv(datos, group = municipio, v = 5) # S4
res <- tune_grid(workflow(rec_i, ridge), folds,
  grid = 30, metrics = metric_set(rmse, rsq))
```

- La red no se entrena: solo ve. Las 2.048 columnas son las variables derivadas de la Parte 1; el Ridge y la validación por municipios son las sesiones 4 y 5 sin cambios. Si sobran datos, se descongelan las últimas capas y se reentrena con tasa de aprendizaje pequeña.

Presentación estudiantil: el paper de la sesión 7

Quince minutos, tres preguntas

Bluwstein, Buckmann, Joseph, Kapadia & Şimşek (2023) o Ludwig & Mullainathan (2024), a elección de quien presenta.

- 1 **¿Qué se predice y por qué importa?** Crisis financieras con uno o dos años de anticipación en 17 países desde 1870; o la decisión del juez de encarcelar, a partir de la foto del acusado, como punto de partida para generar hipótesis.
- 2 **¿Con qué datos y método, y cómo se evalúa?** Qué familias comparan contra la logística; cómo validan con eventos raros y dependencia temporal; qué métrica y qué umbral fijan.
- 3 **¿Qué se encuentra y cómo se interpreta?** Qué herramienta de la sesión 7 abre la caja (Shapley; morfología de rostros con sujetos humanos) y qué afirman y *qué no* a partir de ella.

Presentación estudiantil: el paper de la sesión 7

Quince minutos, tres preguntas

Bluwstein, Buckmann, Joseph, Kapadia & Şimşek (2023) o Ludwig & Mullainathan (2024), a elección de quien presenta.

- 1 **¿Qué se predice y por qué importa?** Crisis financieras con uno o dos años de anticipación en 17 países desde 1870; o la decisión del juez de encarcelar, a partir de la foto del acusado, como punto de partida para generar hipótesis.
- 2 **¿Con qué datos y método, y cómo se evalúa?** Qué familias comparan contra la logística; cómo validan con eventos raros y dependencia temporal; qué métrica y qué umbral fijan.
- 3 **¿Qué se encuentra y cómo se interpreta?** Qué herramienta de la sesión 7 abre la caja (Shapley; morfología de rostros con sujetos humanos) y qué afirman y *qué no* a partir de ella.

Para el resto de la sala

¿Las explicaciones se calculan fuera del entrenamiento? ¿Alguna conclusión trata una contribución de Shapley como un efecto? ¿Qué papel juega la no linealidad (crédito $\times$ curva de rendimientos) en la ganancia sobre la logística?

- 1 Qué es una red neuronal
- 2 Cómo se ajusta: gradiente, retropropagación y regularización
- 3 Cuándo usar redes, y cuándo no
- 4 Redes convolucionales: la aplicación con imágenes
- 5 Aplicación guiada: una red contra los árboles, y una CNN que ve mesas
- 6 Interpretación, texto como lectura y cierre

Qué dice la red y qué no dice

- **Las unidades ocultas no significan nada por sí solas:** no hay coeficientes que leer. El kit de la sesión 7 sí funciona: importancia por permutación, PDP y ALE, SHAP por muestreo (KernelSHAP) o por gradientes. Todo fuera del entrenamiento y con semillas.

Qué dice la red y qué no dice

- **Las unidades ocultas no significan nada por sí solas:** no hay coeficientes que leer. El kit de la sesión 7 sí funciona: importancia por permutación, PDP y ALE, SHAP por muestreo (KernelSHAP) o por gradientes. Todo fuera del entrenamiento y con semillas.
- **En imágenes, la saliencia:** qué píxeles mueven la predicción (Grad-CAM). Útil para auditar (¿mira los techos o la marca de agua de la foto?), peligrosa para narrar.

Qué dice la red y qué no dice

- **Las unidades ocultas no significan nada por sí solas:** no hay coeficientes que leer. El kit de la sesión 7 sí funciona: importancia por permutación, PDP y ALE, SHAP por muestreo (KernelSHAP) o por gradientes. Todo fuera del entrenamiento y con semillas.
- **En imágenes, la saliencia:** qué píxeles mueven la predicción (Grad-CAM). Útil para auditar (¿mira los techos o la marca de agua de la foto?), peligrosa para narrar.
- **El proxy y el dominio:** un R^2 de 0,5 contra la encuesta es una medida de cuánto se ve desde arriba, no una licencia para reemplazar la encuesta. Fuera del país o de la década de entrenamiento, el modelo se degrada sin avisar.

Qué dice la red y qué no dice

- **Las unidades ocultas no significan nada por sí solas:** no hay coeficientes que leer. El kit de la sesión 7 sí funciona: importancia por permutación, PDP y ALE, SHAP por muestreo (KernelSHAP) o por gradientes. Todo fuera del entrenamiento y con semillas.
- **En imágenes, la saliencia:** qué píxeles mueven la predicción (Grad-CAM). Útil para auditar (¿mira los techos o la marca de agua de la foto?), peligrosa para narrar.
- **El proxy y el dominio:** un R^2 de 0,5 contra la encuesta es una medida de cuánto se ve desde arriba, no una licencia para reemplazar la encuesta. Fuera del país o de la década de entrenamiento, el modelo se degrada sin avisar.
- **La tabla decide:** si la red no le gana al bosque en la prueba, la respuesta a “¿usamos *deep learning*?” es no, y es una buena respuesta.

Qué dice la red y qué no dice

- **Las unidades ocultas no significan nada por sí solas:** no hay coeficientes que leer. El kit de la sesión 7 sí funciona: importancia por permutación, PDP y ALE, SHAP por muestreo (KernelSHAP) o por gradientes. Todo fuera del entrenamiento y con semillas.
- **En imágenes, la saliencia:** qué píxeles mueven la predicción (Grad-CAM). Útil para auditar (¿mira los techos o la marca de agua de la foto?), peligrosa para narrar.
- **El proxy y el dominio:** un R^2 de 0,5 contra la encuesta es una medida de cuánto se ve desde arriba, no una licencia para reemplazar la encuesta. Fuera del país o de la década de entrenamiento, el modelo se degrada sin avisar.
- **La tabla decide:** si la red no le gana al bosque en la prueba, la respuesta a “¿usamos *deep learning*?” es no, y es una buena respuesta.

Y, no β , versión redes

Una CNN que predice riqueza desde el cielo mide cuánta información sobre el bienestar contiene el paisaje construido. No dice que construir techos de zinc enriquezca a nadie. La frontera causal es la sesión 9.

Texto y modelos de lenguaje: la lectura complementaria

Fuera del núcleo del curso; para los proyectos que lo necesiten

- **El texto como datos** (Gentzkow, Kelly & Taddy, 2019): contar palabras (bolsa de palabras, TF-IDF) convierte un discurso o un informe en una fila con miles de columnas; desde ahí aplica todo el curso: Lasso, bosques, PCA.

Texto y modelos de lenguaje: la lectura complementaria

Fuera del núcleo del curso; para los proyectos que lo necesiten

- **El texto como datos** (Gentzkow, Kelly & Taddy, 2019): contar palabras (bolsa de palabras, TF-IDF) convierte un discurso o un informe en una fila con miles de columnas; desde ahí aplica todo el curso: Lasso, bosques, PCA.
- **Embeddings**: cada palabra como un vector aprendido de su contexto; el sentido como geometría. La misma idea que la penúltima capa de la CNN de hoy: una representación reutilizable.

Texto y modelos de lenguaje: la lectura complementaria

Fuera del núcleo del curso; para los proyectos que lo necesiten

- **El texto como datos** (Gentzkow, Kelly & Taddy, 2019): contar palabras (bolsa de palabras, TF-IDF) convierte un discurso o un informe en una fila con miles de columnas; desde ahí aplica todo el curso: Lasso, bosques, PCA.
- **Embeddings**: cada palabra como un vector aprendido de su contexto; el sentido como geometría. La misma idea que la penúltima capa de la CNN de hoy: una representación reutilizable.
- **Transformers y LLMs**: representaciones *en contexto* aprendidas sobre billones de palabras. Para un economista, su uso más sólido es como **instrumento de medición** (clasificar, extraer, codificar) validado contra una muestra etiquetada a mano, con la disciplina de la sesión 7 (Dell, 2025; Korinek, 2023).

Texto y modelos de lenguaje: la lectura complementaria

Fuera del núcleo del curso; para los proyectos que lo necesiten

- **El texto como datos** (Gentzkow, Kelly & Taddy, 2019): contar palabras (bolsa de palabras, TF-IDF) convierte un discurso o un informe en una fila con miles de columnas; desde ahí aplica todo el curso: Lasso, bosques, PCA.
- **Embeddings**: cada palabra como un vector aprendido de su contexto; el sentido como geometría. La misma idea que la penúltima capa de la CNN de hoy: una representación reutilizable.
- **Transformers y LLMs**: representaciones *en contexto* aprendidas sobre billones de palabras. Para un economista, su uso más sólido es como **instrumento de medición** (clasificar, extraer, codificar) validado contra una muestra etiquetada a mano, con la disciplina de la sesión 7 (Dell, 2025; Korinek, 2023).

Para el proyecto

Si el proyecto usa texto, la regla no cambia: partición, *pipeline*, *baseline*, prueba una vez, interpretación con advertencias. Cambia la representación; la evaluación es idéntica.

Cuándo usar qué, y qué leer

	Usar cuando	Cuidado con
Lineal regularizado	señal suave; pocos datos; hay que explicar	no ve lo que no se escribe
Árboles y <i>boosting</i>	tabular mediano con columnas con sentido	extrapolación; afinado del <i>boosting</i>
Red densa	tabular muy grande, o como comparación	escala, semillas, η ; rara vez gana
CNN preentrenada + Ridge	imágenes con miles de etiquetas	dominio distinto a ImageNet; proxy
Red desde cero	millones de ejemplos y GPU	todo lo anterior, multiplicado
Texto y LLM	medir a partir de documentos	validar contra etiquetas humanas

Cuándo usar qué, y qué leer

	Usar cuando	Cuidado con
Lineal regularizado	señal suave; pocos datos; hay que explicar	no ve lo que no se escribe
Árboles y <i>boosting</i>	tabular mediano con columnas con sentido	extrapolación; afinado del <i>boosting</i>
Red densa	tabular muy grande, o como comparación	escala, semillas, η ; rara vez gana
CNN preentrenada + Ridge	imágenes con miles de etiquetas	dominio distinto a ImageNet; proxy
Red desde cero	millones de ejemplos y GPU	todo lo anterior, multiplicado
Texto y LLM	medir a partir de documentos	validar contra etiquetas humanas

Qué leer: ISL cap. 10 (10.1 a 10.3 y 10.6 a 10.7; 10.8 por curiosidad); Jean et al. (2016) con su material suplementario, por el diseño del *transfer learning*; Grinsztajn et al. (2022) por la evidencia tabular; Dell (2025) como mapa del *deep learning* para economistas; Goodfellow, Bengio & Courville (2016), caps. 6 a 9, para quien quiera el fondo.

Recap

- 1 **Una red** es una composición de regresiones con activaciones no lineales; cada capa oculta es un conjunto de variables derivadas aprendidas para predecir. Aproxima todo; por eso sobreajusta todo.
- 2 **Se entrena** con SGD por mini-lotes (Adam) sobre una pérdida no convexa que en la práctica se deja optimizar; la **retropropagación** es la regla de la cadena con reutilización: el gradiente cuesta una ida y una vuelta.
- 3 **Se regulariza** con el kit de siempre: *weight decay* (Ridge), *early stopping* (la U en vivo), *dropout* (ensamble implícito), más datos. Escala dentro del *pipeline*; semillas promediadas.
- 4 **Cuándo:** en tabular mediano los árboles ganan o empatan (Grinsztajn et al.; ISL 10.6); la red paga con estructura, escala o representaciones transferibles.
- 5 **CNN:** localidad y pesos compartidos; bordes $\rightarrow$ partes $\rightarrow$ objetos; **transfer learning** convierte la visión en un paso de preprocesamiento. Jean et al.: 37 a 55 % del consumo desde el cielo, con un proxy que hay que validar.
- 6 **La tabla decide**, también aquí: misma partición, misma prueba, intervalos y semillas.

Preparación para la próxima clase...

Para aprovechar al máximo la próxima sesión

Lecturas obligatorias de esta semana:

- ISL, cap. 10, secciones 10.1 a 10.3 y 10.6 a 10.7.
Guía: 10.1 y 10.2 son la Parte 1; 10.7 es la Parte 2; 10.3.5 es la ResNet de la aplicación.
- Jean et al. (2016), *Combining Satellite Imagery and Machine Learning to Predict Poverty, Science*.
Guía: es corto; concéntrense en la figura 1 y en el diseño en dos etapas.

Para la presentación estudiantil de la sesión 9: Jean et al. (2016) o Glaeser, Kincaid & Naik (2018), *Computer Vision and Real Estate*, NBER WP 25174.

Para aprovechar al máximo la próxima sesión

Lecturas obligatorias de esta semana:

- ISL, cap. 10, secciones 10.1 a 10.3 y 10.6 a 10.7.
Guía: 10.1 y 10.2 son la Parte 1; 10.7 es la Parte 2; 10.3.5 es la ResNet de la aplicación.
- Jean et al. (2016), *Combining Satellite Imagery and Machine Learning to Predict Poverty, Science*.
Guía: es corto; concéntrense en la figura 1 y en el diseño en dos etapas.

Para la presentación estudiantil de la sesión 9: Jean et al. (2016) o Glaeser, Kincaid & Naik (2018), *Computer Vision and Real Estate*, NBER WP 25174.

Próxima sesión, la última: estructura sin Y (PCA como índice y k -means como segmentación, y cómo se validan sin variable de respuesta); la frontera causal (doble selección, *double machine learning*, *causal forests*: cuándo la predicción ayuda a estimar un β); el curso en una diapositiva; y las **presentaciones de los proyectos**. Lectura previa: ISL 12.2 y 12.4; Athey & Imbens (2019).

Para aprovechar al máximo la próxima sesión

Lecturas obligatorias de esta semana:

- ISL, cap. 10, secciones 10.1 a 10.3 y 10.6 a 10.7.
Guía: 10.1 y 10.2 son la Parte 1; 10.7 es la Parte 2; 10.3.5 es la ResNet de la aplicación.
- Jean et al. (2016), *Combining Satellite Imagery and Machine Learning to Predict Poverty, Science*.
Guía: es corto; concéntrense en la figura 1 y en el diseño en dos etapas.

Para la presentación estudiantil de la sesión 9: Jean et al. (2016) o Glaeser, Kincaid & Naik (2018), *Computer Vision and Real Estate*, NBER WP 25174.

Próxima sesión, la última: estructura sin Y (PCA como índice y k -means como segmentación, y cómo se validan sin variable de respuesta); la frontera causal (doble selección, *double machine learning*, *causal forests*: cuándo la predicción ayuda a estimar un β); el curso en una diapositiva; y las **presentaciones de los proyectos**. Lectura previa: ISL 12.2 y 12.4; Athey & Imbens (2019). **Aplicación de esta semana en R:** la red densa contra los árboles en la misma prueba, con semillas; la CNN preentrenada sobre las imágenes de las mesas, con validación por municipios.

Referencias esenciales

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. ed.). Springer. Cap. 10. [ISL]
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Caps. 6–9.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521, 436–444.
- Kingma, D., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
- Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929–1958.
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? *NeurIPS Datasets and Benchmarks*.
- Jean, N., Burke, M., Xie, M., Davis, W., Lobell, D., & Ermon, S. (2016). Combining Satellite Imagery and Machine Learning to Predict Poverty. *Science*, 353(6301), 790–794.
- Yeh, C., et al. (2020). Using Publicly Available Satellite Imagery and Deep Learning to Understand Economic Well-Being in Africa. *Nature Communications*, 11, 2583.
- Henderson, V., Storeygard, A., & Weil, D. (2012). Measuring Economic Growth from Outer Space. *AER*, 102(2), 994–1028.
- Glaeser, E., Kincaid, M., & Naik, N. (2018). Computer Vision and Real Estate. NBER Working Paper 25174.
- Gentzkow, M., Kelly, B., & Taddy, M. (2019). Text as Data. *Journal of Economic Literature*, 57(3), 535–574.
- Dell, M. (2025). Deep Learning for Economists. *Journal of Economic Literature*, 63(1), 5–58.
- Bluwstein, K., et al. (2023). Credit Growth, the Yield Curve and Financial Crisis Prediction. *Journal of International Economics*, 145, 103773.
- Ludwig, J., & Mullainathan, S. (2024). Machine Learning as a Tool for Hypothesis Generation. *QJE*, 139(2), 751–827.

7 Apéndice: para profundizar

A1. Retropropagación: derivación con una capa oculta

Setup (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

A1. Retropropagación: derivación con una capa oculta

Setup (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

1. Capa de salida (primer eslabón de la cadena):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{el residuo}).$$

A1. Retropropagación: derivación con una capa oculta

Setup (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

1. Capa de salida (primer eslabón de la cadena):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{el residuo}).$$

2. Gradientes de la capa de salida:

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}.$$

A1. Retropropagación: derivación con una capa oculta

Setup (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

1. Capa de salida (primer eslabón de la cadena):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{el residuo}).$$

2. Gradientes de la capa de salida:

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}.$$

3. Propagar el error hacia atrás (por cada unidad oculta, la cadena pasa por $\hat{y}$ y por g):

$$\delta^{(1)} = (w^{(2)} \delta^{(2)}) \odot g'(z).$$

A1. Retropropagación: derivación con una capa oculta

Setup (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

1. Capa de salida (primer eslabón de la cadena):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{el residuo}).$$

2. Gradientes de la capa de salida:

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}.$$

3. Propagar el error hacia atrás (por cada unidad oculta, la cadena pasa por $\hat{y}$ y por g):

$$\delta^{(1)} = (w^{(2)} \delta^{(2)}) \odot g'(z).$$

4. Gradientes de la capa oculta:

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x', \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

A1. Retropropagación: derivación con una capa oculta

Setup (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

1. Capa de salida (primer eslabón de la cadena):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{el residuo}).$$

2. Gradientes de la capa de salida:

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}.$$

3. Propagar el error hacia atrás (por cada unidad oculta, la cadena pasa por $\hat{y}$ y por g):

$$\delta^{(1)} = (w^{(2)} \delta^{(2)}) \odot g'(z).$$

4. Gradientes de la capa oculta:

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x', \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

Con ReLU, $g'(z) = \mathbb{I}\{z > 0\}$: el error solo fluye por las unidades activas. Con más capas, el paso 3 se repite capa por capa. Un paso con números es material del Problem Set 2.

A2. Momento y Adam

El SGD puro oscila en valles estrechos y avanza despacio en mesetas. Dos remedios que se combinan:

A2. Momento y Adam

El SGD puro oscila en valles estrechos y avanza despacio en mesetas. Dos remedios que se combinan: **Momento**: acumular una media móvil del gradiente y moverse en su dirección,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad \theta_t = \theta_{t-1} - \eta m_t.$$

A2. Momento y Adam

El SGD puro oscila en valles estrechos y avanza despacio en mesetas. Dos remedios que se combinan: **Momento**: acumular una media móvil del gradiente y moverse en su dirección,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad \theta_t = \theta_{t-1} - \eta m_t.$$

Adam (Kingma & Ba, 2015): además, una media móvil del gradiente al cuadrado para *escalar* el paso de cada parámetro por su propia volatilidad,

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$
$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

A2. Momento y Adam

El SGD puro oscila en valles estrechos y avanza despacio en mesetas. Dos remedios que se combinan: **Momento**: acumular una media móvil del gradiente y moverse en su dirección,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad \theta_t = \theta_{t-1} - \eta m_t.$$

Adam (Kingma & Ba, 2015): además, una media móvil del gradiente al cuadrado para *escalar* el paso de cada parámetro por su propia volatilidad,

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

- Los *defaults* $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\epsilon = 10^{-8}$ casi nunca se tocan; $\eta \approx 10^{-3}$ es el punto de partida. Las correcciones $1 - \beta^t$ compensan que $m_0 = v_0 = 0$.
- Parámetros con gradientes grandes reciben pasos chicos y viceversa: una estandarización implícita del paisaje. Adam tolera predictores mal escalados mejor que SGD, pero no tan bien como normalizar. AdamW resta el *weight decay* aparte del gradiente.

A3. La aritmética de una capa convolucional

Tamaño de salida. Una imagen de $n \times n$, un filtro de $k \times k$, relleno (*padding*) p y paso (*stride*) s producen un mapa de

$$\left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 \times \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Con 224×224 , $k = 3$, $p = 1$, $s = 1$: 224×224 . Un *pooling* 2×2 lo baja a 112×112 .

A3. La aritmética de una capa convolucional

Tamaño de salida. Una imagen de $n \times n$, un filtro de $k \times k$, relleno (*padding*) p y paso (*stride*) s producen un mapa de

$$\left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 \times \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Con 224×224 , $k = 3$, $p = 1$, $s = 1$: 224×224 . Un *pooling* 2×2 lo baja a 112×112 . **Parámetros.** Una capa con F filtros de $k \times k$ sobre una entrada con C canales tiene $F(k^2C + 1)$ pesos, *independientemente del tamaño de la imagen*. Sesenta y cuatro filtros 3×3 sobre RGB: $64 \times (27 + 1) = 1.792$. La capa densa equivalente sobre $224 \times 224 \times 3$ entradas y 64 salidas tendría $64 \times 150.529 \approx 9,6$ millones.

A3. La aritmética de una capa convolucional

Tamaño de salida. Una imagen de $n \times n$, un filtro de $k \times k$, relleno (*padding*) p y paso (*stride*) s producen un mapa de

$$\left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 \times \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Con 224×224 , $k = 3$, $p = 1$, $s = 1$: 224×224 . Un *pooling* 2×2 lo baja a 112×112 . **Parámetros.** Una capa con F filtros de $k \times k$ sobre una entrada con C canales tiene $F(k^2C + 1)$ pesos, *independientemente del tamaño de la imagen*. Sesenta y cuatro filtros 3×3 sobre RGB: $64 \times (27 + 1) = 1.792$. La capa densa equivalente sobre $224 \times 224 \times 3$ entradas y 64 salidas tendría $64 \times 150.529 \approx 9,6$ millones. **Cómputo.** Cada posición del mapa cuesta k^2C multiplicaciones; la capa completa, $F \cdot k^2C \cdot (\text{posiciones})$. Los pesos son pocos; las operaciones, muchas: por eso las GPU.

A3. La aritmética de una capa convolucional

Tamaño de salida. Una imagen de $n \times n$, un filtro de $k \times k$, relleno (*padding*) p y paso (*stride*) s producen un mapa de

$$\left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 \times \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Con 224×224 , $k = 3$, $p = 1$, $s = 1$: 224×224 . Un *pooling* 2×2 lo baja a 112×112 . **Parámetros.** Una capa con F filtros de $k \times k$ sobre una entrada con C canales tiene $F(k^2C + 1)$ pesos, *independientemente del tamaño de la imagen*. Sesenta y cuatro filtros 3×3 sobre RGB: $64 \times (27 + 1) = 1.792$. La capa densa equivalente sobre $224 \times 224 \times 3$ entradas y 64 salidas tendría $64 \times 150.529 \approx 9,6$ millones. **Cómputo.** Cada posición del mapa cuesta k^2C multiplicaciones; la capa completa, $F \cdot k^2C \cdot (\text{posiciones})$. Los pesos son pocos; las operaciones, muchas: por eso las GPU.

- **ResNet50:** unos 25 millones de parámetros y 50 capas con “atajos” que suman la entrada de un bloque a su salida, lo que permite entrenar redes profundas sin que el gradiente se desvanezca. Su penúltima capa, tras el *pooling* global, tiene 2.048 números por imagen: las variables derivadas de la aplicación.

A4. Aproximación universal: qué dice y qué no

Enunciado (Cybenko, 1989; Hornik, 1991). Sea g una función de activación no polinómica (sigmoide, ReLU). Para toda función continua f sobre un compacto de $\mathbb{R}^p$ y todo $\varepsilon > 0$ existen K y pesos tales que

$$\sup_x \left| f(x) - \sum_{k=1}^K \beta_k g(w'_k x + b_k) \right| < \varepsilon.$$

A4. Aproximación universal: qué dice y qué no

Enunciado (Cybenko, 1989; Hornik, 1991). Sea g una función de activación no polinómica (sigmoide, ReLU). Para toda función continua f sobre un compacto de $\mathbb{R}^p$ y todo $\varepsilon > 0$ existen K y pesos tales que

$$\sup_x \left| f(x) - \sum_{k=1}^K \beta_k g(w'_k x + b_k) \right| < \varepsilon.$$

- **Es un resultado de densidad**, como el de Weierstrass para polinomios. No dice cuántas unidades (K puede crecer exponencialmente con p y $1/\varepsilon$), ni que SGD las encuentre, ni nada sobre generalización.
- **Profundidad**: hay funciones que una red profunda representa con $O(p)$ unidades y una red de una capa necesita $O(2^p)$ (Telgarsky, 2016): la composición reutiliza piezas. Es la razón teórica de “profundo y no ancho”, y coincide con lo que se ve en las CNN.
- **La analogía con la sesión 2**: los *splines* también aproximan cualquier función suave; la red aprende *dónde* poner los nodos (w_k, b_k) y en qué combinaciones de variables. Con p grande, esa es la diferencia entre lo posible y lo imposible.